

# ADT List

IF2110/IF2111 – Algoritma dan Struktur Data  
Sekolah Teknik Elektro dan Informatika  
Institut Teknologi Bandung

# ADT List

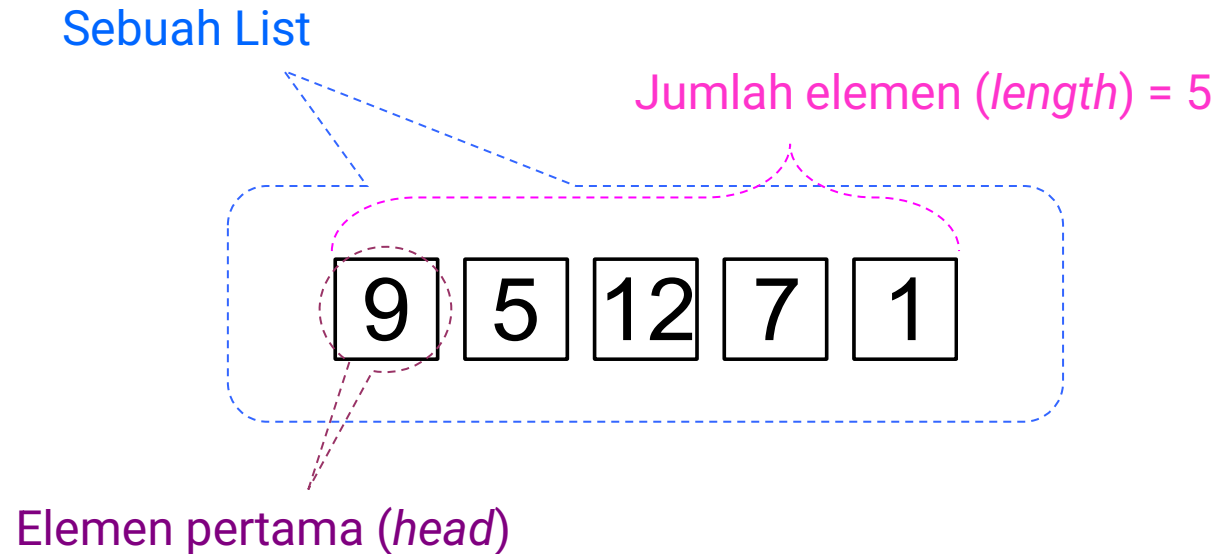
*List*, dikenal juga dengan *sequence*, merupakan sekumpulan elemen **bertipe sama** yang memiliki suatu **keterurutan tertentu** (*ordered*, tidak harus *sorted*).

Ada konsep elemen pertama, elemen kedua, elemen ke- $n$ .

Contoh:

- Daftar belanjaan, terurut berdasarkan urutan pada saat menulis di kertas.
- Daftar kota yang disinggahi saat *road trip*, terurut berdasarkan rute perjalanan.
- *Instagram feed*, sebuah daftar “postingan” yang terurut berdasarkan waktu publikasi (*not anymore?*).
- *String* adalah *list of characters*!

# Istilah-istilah



- *head*: elemen pertama *list*
- *length*: jumlah elemen sebuah *list*
- *List* kosong (*empty list*): *list* yang tidak mempunyai elemen
- Traversal: mengunjungi elemen *list* satu-per-satu dari ujung sampai ujung satunya

# Ingat kembali...

ADT adalah pemodelan suatu tipe data yang didefinisikan perilakunya berdasarkan:

**data** yang terkandung di dalamnya,

**himpunan nilai yang mungkin dimiliki** oleh data tersebut, serta

**operasi yang dapat diterapkan** terhadap data tersebut.

Renungkan **data**, **himpunan nilai**, dan **operasi** untuk ADT List!

# Struktur data untuk ADT List

ADT List dapat diimplementasikan dengan struktur data *array* maupun berkait.

- Implementasi dengan *array*

Terdapat beberapa variasi:

- Elemen disimpan secara kontigu
- Elemen disimpan secara tersebar di dalam *array*

Batasan jumlah elemen?

- Menggunakan *array* statik: ada istilah *list* penuh.
- Menggunakan *array* dinamis?

- Implementasi berkait

Akan dibahas di minggu ke-7.

# Operasi ADT List (1/2)

Memeriksa apakah list kosong (**isEmpty**)

Memeriksa apakah sebuah nilai ada di dalam list/searching  
→ biasanya berujung pada “nilai x ada di indeks berapa?” (**indexOf**)

Ada berapa elemen dalam list (**length**)

Melihat nilai pada posisi tertentu (**getElmt**)

Mengubah nilai pada posisi tertentu (**setElmt**)

Menggabungkan dua list (**concat**)

# Operasi ADT List (2/2)

Menambahkan elemen (**insert-**)

- Di awal list
- Di akhir list
- Pada posisi/indeks tertentu

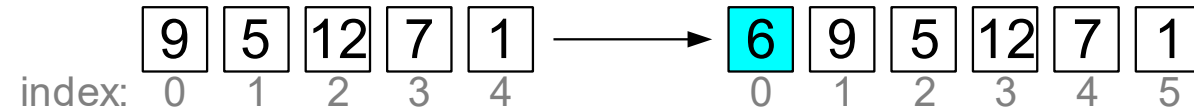
Menghapus elemen (**delete-**)

- Di awal list
- Di akhir list
- Pada posisi/indeks tertentu

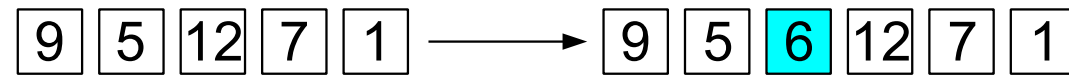
Pola **traversal** (mengunjungi elemen satu per satu)

# Ilustrasi: Insert dan Delete

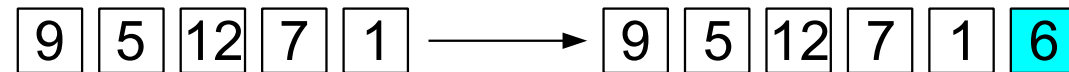
Insert 6 first:



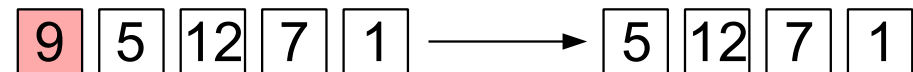
Insert 6 after 5 or insert 6 at index=2:



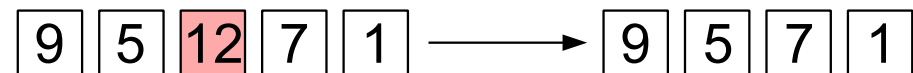
Insert 6 last:



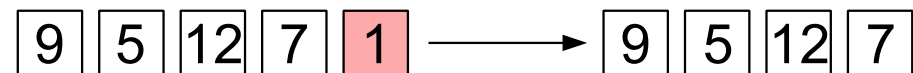
Delete first element:



Delete 12 or delete element at index=2:



Delete last element:





# Implementasi ADT List dengan Array (elemen kontigu)

IF2110/IF2111 – Algoritma dan Struktur Data  
Sekolah Teknik Elektro dan Informatika  
Institut Teknologi Bandung

# Implementasi ADT List dengan Array

Alat dan bahan:

- 1) Sebuah *array* dengan ukuran tertentu.
- 2) Cara untuk mengetahui berapa elemen yang sedang terisi & elemen mana saja yang sudah terisi:
  - alt-1: implisit – beri nilai khusus untuk elemen yang sedang kosong.
  - alt-2: eksplisit – simpan jumlah elemen efektif.

# Implisit vs. eksplisit

## Implisit (alt-1)

*Array* kosong harus diinisialisasi memorinya dengan nilai khusus yang disebut *mark*.

Elemen terisi harus kontigu: tidak boleh ada *mark* di antara nilai terisi.

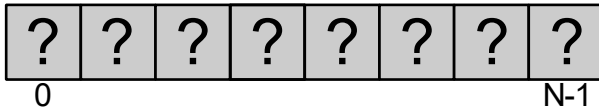
## Eksplisit (alt-2)

Semua elemen *array* dapat berisi nilai yang valid namun yang dipedulikan hanya elemen pada indeks-indeks yang efektif ( $0..n_{\text{Eff}}-1$ ).

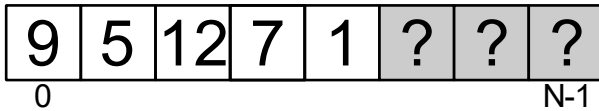
# Implisit vs. eksplisit: ilustrasi

## Implisit (alt-1)

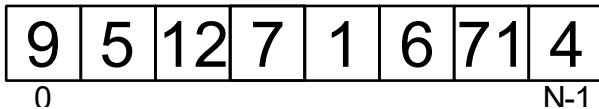
Contoh *array* kosong:



Contoh *array* terisi sebagian:

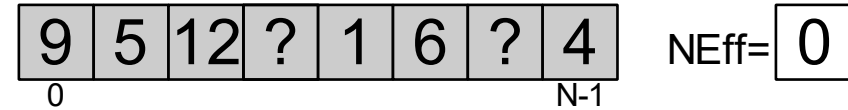


Contoh *array* penuh:

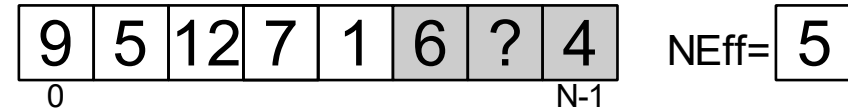


## Eksplisit (alt-2)

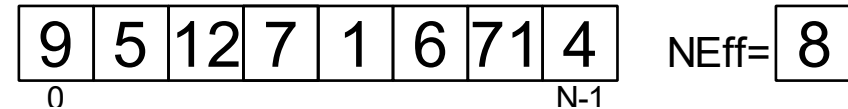
Contoh *array* kosong:



Contoh *array* terisi sebagian:



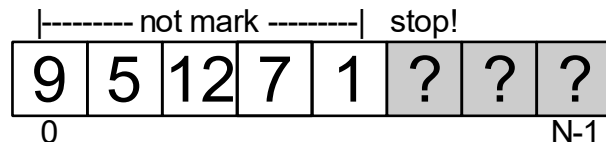
Contoh *array* penuh:



# Implisit vs. eksplisit: karakteristik operasi

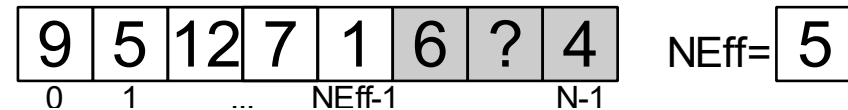
## Implisit (alt-1)

Menggunakan pola “*while* belum ketemu *mark*”



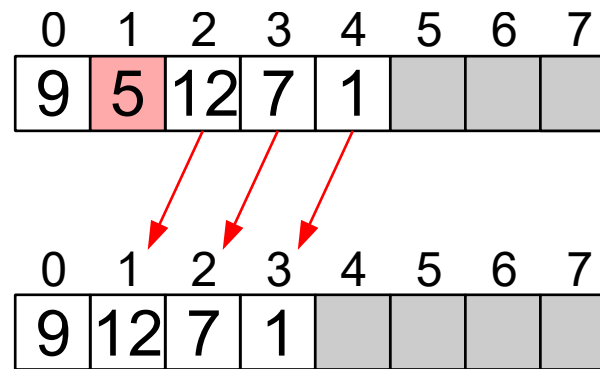
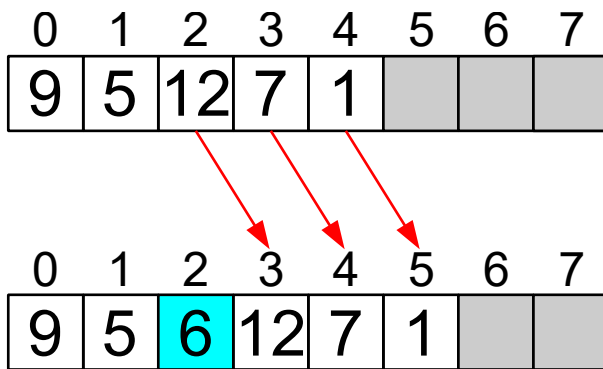
## Eksplisit (alt-2)

Menggunakan pola traversal dari indeks 0 sampai  $n_{\text{Eff}}-1$



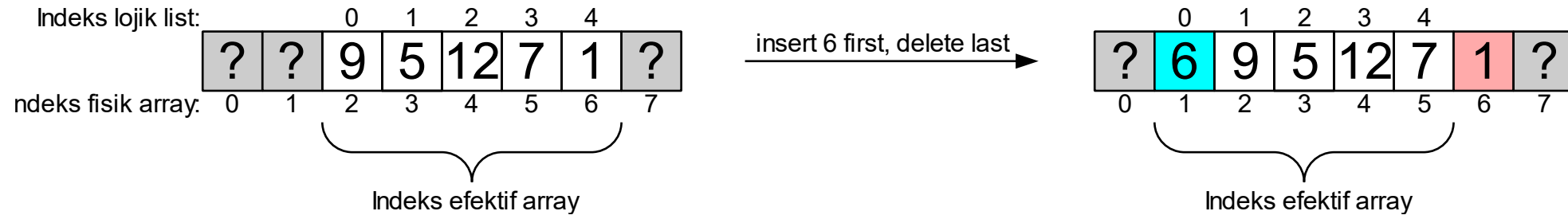
# Insert & delete

Membutuhkan penggeseran elemen-elemen setelah posisi penambahan/ penghapusan elemen → *insert first* perlu menggeser nEff buah elemen.



Bagaimana agar lebih efisien?

# Ide: elemen tidak harus “rata kiri”



- *Insert first/last* tidak perlu geser elemen jika masih ada ruang kosong di depan/belakang.
- *Delete first/last* tidak perlu geser elemen.

Indeks (fisik) efektif pertama?

- alt-1: skip semua elemen yang bernilai *mark*.
- alt-2: perlu menyimpan informasi indeks pertama.

# Ringkasan: 4 alternatif

alt-1a: implisit, rata kiri

alt-2a: eksplisit, rata kiri

alt-1b: implisit, tidak rata kiri

alt-2b: eksplisit, tidak rata kiri



# Contoh: rata kiri

(alt-1a, alt-2a)

# Contoh: alt-1a (Notasi Algoritmik)

constant CAPACITY: integer = 100

constant IDX\_UNDEF: integer = -1

constant MARK: integer = -9999

type ElType: integer { *elemen List* }

type List: < contents: array [0..CAPACITY-1] of ElType > { *penyimpanan elemen List.* }

*{ Contoh deklarasi:*

*l: List*

*Contoh akses elemen array pada indeks i:*

*l.contents[i]*

*}*

*{ Konstruktor }*

procedure CreateList(output l: List)

*{ Membentuk List kosong sesuai kapasitas. }*

# Contoh: alt-1a (Notasi Algoritmik)

constant CAPACITY: integer = 100

constant IDX\_UNDEF: integer = -1

constant MARK: integer = -9999

type ElType: integer { *elemen List* }

type List: < contents: array [0..CAPACITY-1] of ElType > { *penyimpanan elemen List.* }

{ *Contoh deklarasi:*

*l: List*

*Contoh akses elemen array pada indeks i:*

*l.contents[i]*

}

{ *Konstruktor* }

procedure CreateList(output l: List)

{ *Membentuk List kosong sesuai kapasitas.* }

Semua elemen diinisialisasi  
dengan *mark*

# Contoh: alt-2a (Notasi Algoritmik)

```
constant CAPACITY: integer = 100  
constant IDX_UNDEF: integer = -1
```

```
type ElType: integer { elemen List }  
type List: < contents: array [0..CAPACITY-1] of ElType { penyimpanan elemen List. }  
          nEff: integer  $\geq$  0 { jumlah elemen efektif List. } >
```

*{ Contoh deklarasi:*

*l: List*

*Contoh akses elemen array pada indeks i:*

*l.contents[i]*

*}*

*{ Konstruktor }*

```
procedure CreateList(output l: List)  
  { Membentuk List kosong sesuai kapasitas. }
```

# Contoh: alt-2a (Notasi Algoritmik)

```
constant CAPACITY: integer = 100  
constant IDX_UNDEF: integer = -1
```

```
type ElType: integer { elemen List }  
type List: < contents: array [0..CAPACITY-1] of ElType { penyimpanan elemen List. }  
          nEff: integer ≥ 0 { jumlah elemen efektif List. } >
```

*{ Contoh deklarasi:*

*l: List*

*Contoh akses elemen array pada indeks i:*

*l.contents[i]*

*}*

*{ Konstruktor }*

```
procedure CreateList(output l: List)  
  { Membentuk List kosong sesuai kapasitas. }
```

nEff diinisialisasi  
dengan 0

# Selektor

```
{ Selektor }  
function isEmpty(l: List) → boolean  
  { Prekondisi: l terdefinisi.  
    Memeriksa apakah l kosong. }  
function length(l: List) → integer  
  { Prekondisi: l terdefinisi.  
    Mengirimkan banyaknya elemen efektif l, 0 jika list kosong. }  
function getElmt(l: List, i: integer): ElType  
  { Prekondisi: l tidak kosong, i di antara 0..length(l).  
    Mengirimkan elemen list l yang ke-i (indeks logik). }  
procedure setElmt(output l: List, input i: integer, input v: ElType)  
  { I.S. l tidak kosong, i di antara 0..length(l).  
    F.S. Elemen l yang ke-i bernilai v.  
    Mengeset nilai elemen list yang ke-i sehingga bernilai v. }
```

# Selektor

```

{ Selektor }
function isEmpty(l: List) → boolean
{ Prekondisi: l terdefinisi.
  Memeriksa apakah list l kosong. }
function isEmpty(l: List) → boolean
{ Prekondisi: l terdefinisi.
  Menghitung jumlah elemen yang bukan bernilai mark. }
function isEmpty(l: List) → boolean
{ Prekondisi: l terdefinisi.
  Menghitung jumlah elemen yang bukan bernilai mark. }
function isEmpty(l: List) → boolean
{ Prekondisi: l tidak kosong, i di antara 0..Length(l).
  Mengirimkan elemen list l yang ke-i (indeks logik). }
procedure setElmt(output l: List, input i: integer, input v: ElType)
{ I.S. l tidak kosong, i di antara 0..Length(l).
  F.S. Elemen l yang ke-i bernilai v.
  Mengeset nilai elemen list yang ke-i sehingga bernilai v. }

```

# Operasi-operasi (1/2)

*{ Operasi-operasi }*

function indexOf(l: List, x: ElType) → integer

*{ Prekondisi: l, x terdefinisi.*

*Mengembalikan indeks elemen pertama l yang bernilai x (jika ada),  
atau mengembalikan IDX\_UNDEF jika tidak ada. }*

procedure insertFirst(input/output l: List, input x: ElType)

*{ I.S. l terdefinisi, mungkin kosong.*

*F.S. x menjadi elemen pertama l. }*

procedure insertAt(input/output l: List, input x: ElType, input idx: integer)

*{ I.S. l terdefinisi, tidak kosong, i merupakan indeks yang valid di l.*

*F.S. x disisipkan dalam l pada indeks ke-i (bukan menempa elemen di i). }*

procedure insertLast(input/output l: List, input x: ElType)

*{ I.S. l terdefinisi, mungkin kosong.*

*F.S. x menjadi elemen terakhir l. }*



# Operasi-operasi (2/2)

*{ Operasi-operasi }*

procedure **deleteFirst**(input/output l: List, output e: ElType)

*{ I.S. l terdefinisi, tidak kosong.*

*F.S. e diset dengan elemen pertama l, elemen pertama l dihapus dari l. }*

procedure **deleteAt**(input/output l: List, input idx: integer, output e: ElType)

*{ I.S. l terdefinisi, tidak kosong, i merupakan indeks yang valid di l.*

*F.F. e diset dengan elemen l pada indeks ke-idx.*

*Elemen l pada indeks ke-idx dihapus dari l. }*

procedure **deleteLast**(input/output l: List, output e: ElType)

*{ I.S. l terdefinisi, tidak kosong.*

*F.S. e diset dengan elemen terakhir l, elemen terakhir l dihapus dari l. }*

function **concat**(l1, l2: List) → List

*{ Prekondisi: l1 dan l2 terdefinisi, mungkin kosong.*

*Mengembalikan hasil Konkatenasi ("Menyambung") dua buah List, l2 ditaruh di belakang l1 }*

# Contoh algoritma: length

```
function length(l: List) → integer
{ ... }
KAMUS LOKAL { untuk alt-1a }
  i: integer
ALGORITMA { untuk alt-1a }
  i ← 0
  while l.contents[i]≠MARK AND i<CAPACITY do
    i ← i+1
  { l.contents[i]=MARK OR i≥CAPACITY }
  → i
```

```
KAMUS LOKAL { untuk alt-2a }
-
ALGORITMA { untuk alt-2a }
  → l.nEff
```

# Contoh algoritma: insertAt

```
procedure insertAt(input/output l: List, input x: ElType, input idx: integer)  
{ ... }
```

KAMUS LOKAL

i: integer

ALGORITMA { untuk alt-1a }

if length(l)<CAPACITY then

  i traversal [length(l)..idx+1]

    l.contents[i] = l.contents[i-1]

  l.contents[idx] = x

ALGORITMA { untuk alt-2a }

if length(l)<CAPACITY then

  i traversal [length(l)..idx+1]

    l.contents[i] = l.contents[i-1]

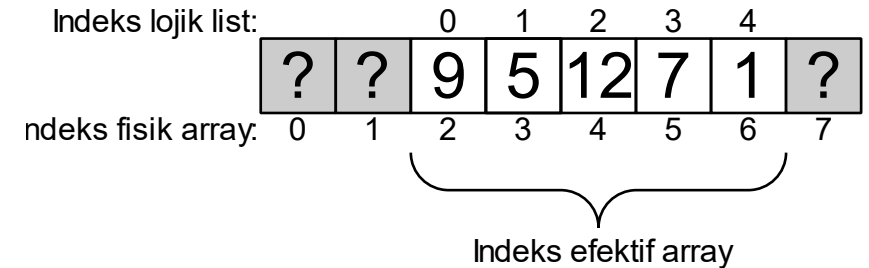
  l.contents[idx] = x

  l.nEff  $\leftarrow$  l.nEff+1

**Contoh: tidak rata kiri  
(alt-1b, alt-2b)**

# Membutuhkan fungsi antara

```
{ Fungsi antara }  
function firstIdx(l: List) → integer  
  { Prekondisi : l tidak kosong  
    Mengirimkan indeks fisik elemen pertama }  
function lastIdx(l: List) → integer  
  { Prekondisi : l tidak kosong  
    Mengirimkan indeks fisik elemen terakhir }  
function isIdxValid(l: List, i: integer) → boolean  
  { Prekondisi: l terdefinisi.  
    Mengirimkan true jika i adalah indeks fisik yang valid utk kapasitas l,  
    yaitu antara 0..MaxEl-1. }  
function isIdxEff(l: List, i: integer) → boolean  
  { Prekondisi: l terdefinisi.  
    Mengirimkan true jika i adalah indeks fisik yang terdefinisi untuk l,  
    yaitu antara firstIdx(l)..lastIdx(l). }
```



# Membutuhkan fungsi antara

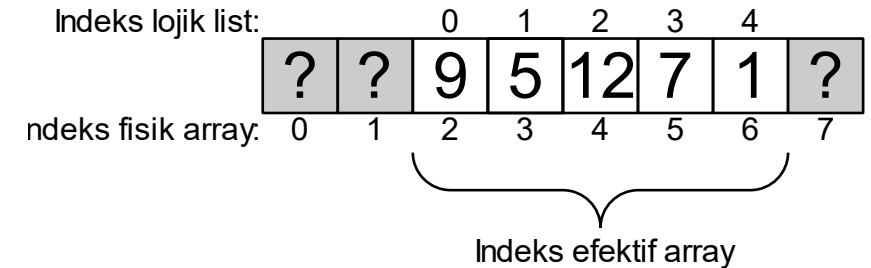
```

{ Fungsi antara }
function firstIdx(l: List) → integer
{ Prekondisi: l tidak kosong
  Mengirimkan true jika i adalah indeks fisik yang valid utk kapasitas l,
  yaitu antara 0..MaxEl-1. }
function lastIdx(l: List) → integer
{ Prekondisi: l tidak kosong
  Mengirimkan true jika i adalah indeks fisik yang terdefinisi untuk l,
  yaitu antara firstIdx(l)..lastIdx(l). }
function isIdxEff(l: List, i: integer) → boolean
{ Prekondisi: l terdefinisi.
  Mengirimkan true jika i adalah indeks fisik yang valid utk kapasitas l,
  yaitu antara 0..MaxEl-1. }
function isIdxEff(l: List, i: integer) → boolean
{ Prekondisi: l terdefinisi.
  Mengirimkan true jika i adalah indeks fisik yang terdefinisi untuk l,
  yaitu antara firstIdx(l)..lastIdx(l). }

```

alt-1b: Cari elemen elemen pertama yang bukan bernilai mark

alt-1b: Cari elemen elemen terakhir yang bukan bernilai mark



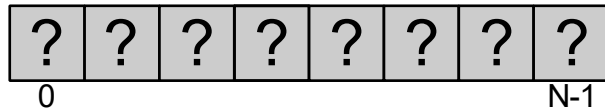
# Implementasi ADT List dengan Array (elemen tersebar)

IF2110/IF2111 – Algoritma dan Struktur Data  
Sekolah Teknik Elektro dan Informatika  
Institut Teknologi Bandung

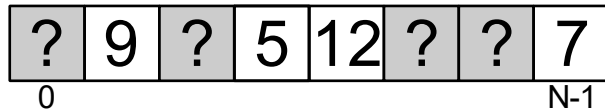
# Elemen list tersebar dalam array (alt-3)

**Ide:** dengan menggunakan nilai khusus (*mark*), elemen sebenarnya tidak perlu disimpan secara kontigu.

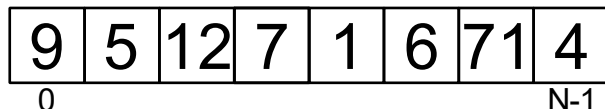
Contoh *array* kosong:



Contoh *array* terisi sebagian:



Contoh *array* penuh:



*Array* kosong harus diinisialisasi dengan *mark*.

Mengunjungi elemen berikutnya harus cek apakah = *mark* (tidak bisa langsung indeks+1).

\*contoh: kapasitas=N



# Operasi-operasi pada alt-3

**isEmpty:** *true* jika semua elemen bernilai khusus (*mark*).

**indexOf:** *skip* elemen-elemen yang bernilai *mark*.

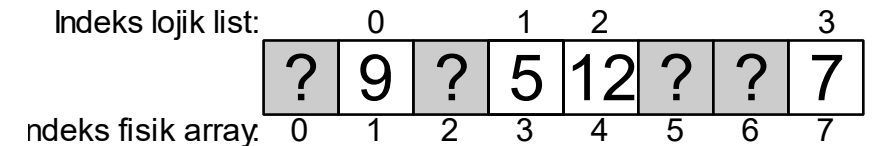
**length:** traversal, cacah yang bukan *mark*.

**getElmt, setElmt:** harus dilakukan mulai indeks fisik = 0 sambil menghitung indeks logik.

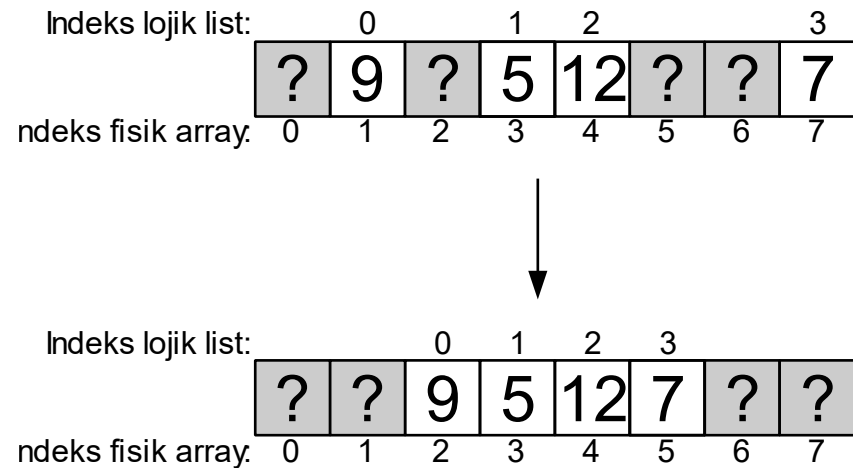
Pola **traversal:** harus dilakukan dari 0 .. kapasitas-1, skip elemen-elemen bernilai *mark*.

**insert:** (sulit??) Paling sederhana: lakukan “defragmen” sebelum insert.

**delete:** set elemen yang hendak dihapus menjadi bernilai *mark*.



# “Defragmen” (memampatkan)



# Contoh algoritma: length

```
function length(l: List) → integer  
{ ... }
```

KAMUS LOKAL

ctr, i: integer

ALGORITMA

ctr ← 0

i traversal [0..CAPACITY-1]

if i≠MARK then

ctr ← ctr+1

→ ctr

# Analisis efisiensi 5 alternatif

alt-1a vs. alt-2a vs. alt-1b vs. alt-2b vs. alt-3

Dari segi penggunaan memori, semuanya sama (sesuai ukuran alokasi di awal).

Dari segi waktu, mana yang lebih baik untuk operasi-operasi berikut:

- insert first
- insert last
- insert di tengah
- delete first
- delete last
- delete di tengah

untuk kasus-kasus *best case*, *worst case*, dan rata-rata?

# Implementasi ADT List dengan Array dinamis

IF2110/IF2111 – Algoritma dan Struktur Data  
Sekolah Teknik Elektro dan Informatika  
Institut Teknologi Bandung

# List dan array dinamis

Implementasi-implementasi ADT List yang telah dibahas memiliki keterbatasan kapasitas sesuai alokasi *array*.

Pada definisi ADT List secara logik tidak mengharuskan ada batasan jumlah elemen.

Bagaimana mengimplementasikan List yang jumlah elemennya tak terbatas menggunakan *array*?

# List dan array dinamis

Menggunakan *array* dinamis:

- Ketika *array* sudah penuh dan hendak dilakukan penambahan elemen, alokasikan *array* baru yang lebih besar (misal  $2 \times$  ukuran yang lama).
- Ketika *array* sudah “sepi” (misal hanya  $\leq 25\%$  terisi), alokasikan *array* baru yang lebih kecil (misal  $0.5 \times$  ukuran yang lama).

# Contoh: alt-2a (rata kiri), dinamis

```
constant INITIAL_CAP: integer = 100
```

```
type ElType: integer { elemen list }
```

```
type List: < contents: array of ElType, { penyimpanan elemen list. }  
           capacity: integer, { dibutuhkan pencatat ukuran alokasi saat ini. }  
           nEff: integer > { pencatat jumlah elemen efektif. }
```

```
{ Konstruktor }
```

```
procedure CreateList(output l: List)
```

```
  { Membentuk List kosong dengan kapasitas awal initialSize. }
```

KAMUS LOKAL

-

ALGORITMA

```
  l.capacity ← INITIAL_CAP
```

```
  alokasi(l.contents, l.capacity)
```

```
  l.nEff ← 0
```



# Contoh: insertAt (alt-2a dinamis)

```
procedure insertAt(input/output l: List, input x: ElType, input idx: integer)  
{ ... }
```

KAMUS LOKAL

tmpArray: array of ElType

i: integer

ALGORITMA

if length(l) < l.capacity then { *masih muat* }

  i traversal [length(l)..idx+1]

    l.contents[i] = l.contents[i-1]

  l.contents[idx] = x

  l.nEff ← l.nEff+1

{ *else: bersambung...* }

# Contoh: insertAt (alt-2a dinamis)

```
else { length(L)=L.capacity (tidak muat) }  
    alokasi(tmpArray, l.capacity)  
    i traversal [0..l.capacity-1]  
        tmpArray[i] ← l.contents[i]  
    dealokasi(l.contents);  
    alokasi(l.contents, l.capacity*2)  
    i traversal [0..idx-1]  
        l.contents[i] ← tmpArray[i]  
    l.contents[idx] ← x  
    i traversal [idx..l.capacity-1]  
        l.contents[i+1] ← tmpArray[i]  
    dealokasi(tmpArray)  
    l.capacity ← l.capacity*2  
    l.nEff ← l.nEff+1
```

*{ Catatan: dalam Bahasa C kita dapat menggunakan fungsi realloc() sehingga tidak perlu menyalin isi array secara manual. Namun ingat, realloc() bisa gagal. }*

# Contoh: deleteAt (alt-2a dinamis)

```
procedure deleteAt(input/output l: List, input idx: integer, output e: ElType)  
{ ... }
```

**KAMUS LOKAL**

tmpArray: array of ElType

newCap, i: integer

**ALGORITMA**

e ← getElmt(l, idx)

if length(l) > l.capacity div 4 then { *belum “sepi”* }

l.nEff ← l.nEff-1

i traversal [idx..length(l)-1]

l.contents[i] ← l.contents[i+1]

{ *else: bersambung...* }

# Contoh: deleteAt (alt-2a dinamis)

```
else { length(L) ≤ L.capacity div 4 (sudah “sepi”) }  
  newCap ← max2(INITIAL_CAP, l.capacity div 2) { mencegah newCap = 0 }  
  alokasi(tmpArray, newCap)  
  i traversal [0..idx-1]  
    tmpArray[i] ← l.contents[i]  
  l.nEff ← l.nEff-1  
  i traversal [idx..length(l)-1]  
    tmpArray[i] ← l.contents[i+1]  
  dealokasi(l.contents)  
  alokasi(l.contents, newCap)  
  i traversal [0..length(l)-1]  
    l.contents[i] ← tmpArray[i]  
  l.capacity ← newCap  
  dealokasi(tmpArray)
```

*{ temukan potensi masalah kinerja! }*

# Latihan Soal

## ADT List dengan Array

# Latihan Soal 1

Asumsikan bahwa sudah didefinisikan list memanfaatkan array statik yang direpresentasikan secara eksplisit dalam notasi algoritmik (alt-2 rata kiri).

Buatlah masing-masing fungsi/prosedur berikut ini untuk representasi struktur data array tersebut dalam notasi algoritmik.

Gunakan sedapat mungkin primitif-primitif lain yang tersedia.

# Latihan Soal 1

```
function isSimetris (l: List) → boolean  
{ Menghasilkan true jika List l simetrik. }  
{ List disebut simetrik jika:  
  - elemen pertama = elemen terakhir,  
  - elemen kedua = elemen sebelum terakhir,  
  - dan seterusnya.  
  List kosong adalah List simetris }
```

```
function plusTab (l1,l2: List) → List  
{ Prekondisi: l1 dan l2 berukuran sama dan tidak kosong. }  
{ Mengirimkan l1+l2, yaitu penjumlahan setiap elemen l1 dan l2  
  pada indeks yang sama (seperti penjumlahan vektor dalam matematika). }
```

```
function countOccurence(l: List, x: ElType) → integer  
{ Menghasilkan berapa banyak kemunculan elemen bernilai x di List l. }  
{ Jika l kosong, menghasilkan 0. }
```

# Latihan Soal 1

function isEqual (l1,l2:List) → boolean

{ Mengirimkan true jika l1 setara dengan l2, yaitu jika ukuran l1 sama dengan ukuran l2 dan semua elemen l1 dan l2 pada indeks yang sama bernilai sama; L1 dan L2 tidak kosong. }

function indexOf (l:List, x:ElType) → IdxType

{ Mencari apakah ada elemen List l yang bernilai x. }  
{ Jika ada, menghasilkan indeks i terkecil, di mana elemen l ke-i = x. Jika tidak ada, mengirimkan indeks tak terdefinisi (idxUndef). Jika list kosong, menghasilkan indeks tak terdefinisi (idxUndef). }  
{ Memakai skema searching tanpa boolean. }



# Latihan Soal 1

```
procedure insertUnique (input/output l:List, input x:ElType)
{ Menambahkan x sebagai elemen terakhir list l,
  pada list dengan elemen unik. }
{ I.S. List l boleh kosong, tetapi tidak penuh
  dan semua elemennya bernilai unik, tidak terurut.
  F.S. Menambahkan x sebagai elemen terakhir l
    jika belum ada elemen yang bernilai x.
    Jika sudah ada elemen list yang bernilai x
    maka I.S. = F.S. dan dituliskan pesan
    “nilai sudah ada”. }
{ Proses : Cek apakah X ada dengan sequential search
  dengan sentinel, kemudian tambahkan jika belum ada. }
```

# Latihan Soal 2

Jika ADT List pada soal 1 diubah menjadi representasi **implisit**, apa yang harus diubah dari struktur data berikut?

```
{  Versi I : Dengan banyaknya elemen secara eksplisit,  
    array statik }  
constant kapasitas: integer = 100  
constant idxUndef: integer = -1 { indeks tak terdefinisi}  
{ Definisi elemen dan koleksi objek }  
type ElType: integer    { type elemen list }  
type List: < ti: array [0..kapasitas-1] of ElType,  
              { memori tempat penyimpan elemen (container) }  
    nEff: integer ≥ 0 { banyaknya elemen efektif } >
```

# Latihan Soal 3

Buatlah fungsi `getFirstIdx` dan `getLastIdx` dalam representasi array secara **eksplisit** dan **implisit**.

Kerjakan kembali fungsi-fungsi yang dikerjakan di latihan soal 1, tetapi untuk representasi implisit. Bagian kode mana saja yang berubah?

# Latihan Soal 4a: ADT Array Eksplisit-Statik

```
procedure closestPair (input l: List; output p1,p2: ElType)
{ I.S.: l terdefinisi, mungkin kosong, p1 dan p2 sembarang. }
{ F.S.:
    Jika l tidak kosong, p1 dan p2 berisi 2 elemen l pada posisi
    berurutan yang memiliki selisih (selalu positif) terkecil.
    Jika kedua elemen nilainya berbeda, maka p1 adalah elemen yang
    bernilai lebih kecil.
    Jika ada beberapa pasang elemen yang memiliki selisih terkecil,
    maka diambil pasangan elemen yang muncul pertama kali.
    Jika l kosong atau hanya terdiri atas 1 elemen, p1 dan p2
    berisi elemen tidak terdefinisi yaitu -999 }
{ Contoh:
    l.ti = [5,3,10,11,20,19];      maka p1=10 dan p2=11
    l.ti = [-2,10,7,30,40,43,9];  maka p1=7  dan p2=10
    l.ti = [-2,10,10,40,40];      maka p1=10 dan p2=10 }
```

# Latihan Soal 4b: ADT Array Eksplisit-Statik

```
function isFront (l1,l2 : List) → boolean
{ Mengembalikan true jika elemen-elemen l1 merupakan bagian awal dari l2 }
{ Contoh: }
{   isFront ([2,3,4], [2,3,4,5,6]) = true }
{   isFront ([2,3,4], [3,4,5,6]) = false }
{   isFront ([], [2,3,4,5,6]) = true }
{   isFront ([2,3,4], [2,3]) = false }
{   isFront ([2,3,4], []) = false }
```